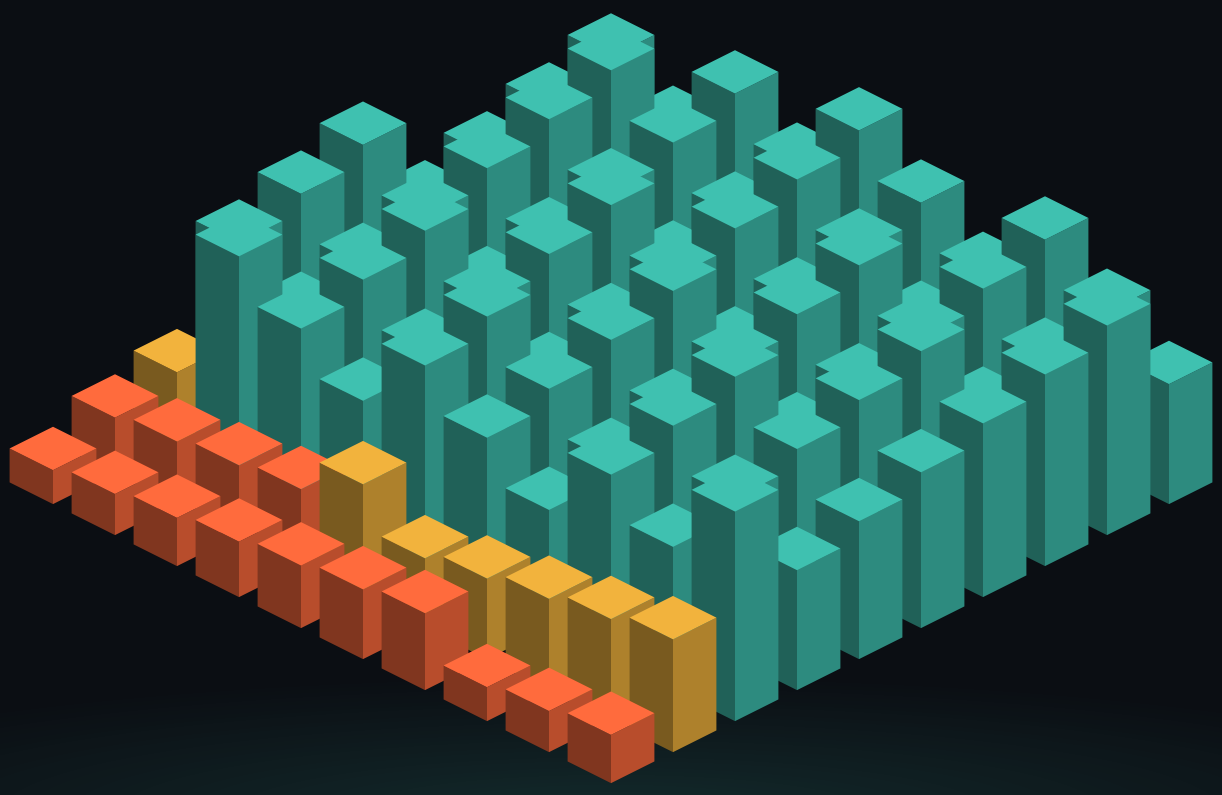
Databricks Free Edition | NASA C-MAPSS FD001

AI-Powered Predictive Maintenance

An end-to-end, audited demo: a medallion pipeline, a remaining-useful-life model with MLflow serving, a 100-engine control-room dashboard and a manuals assistant that cites its sources.



Illustration

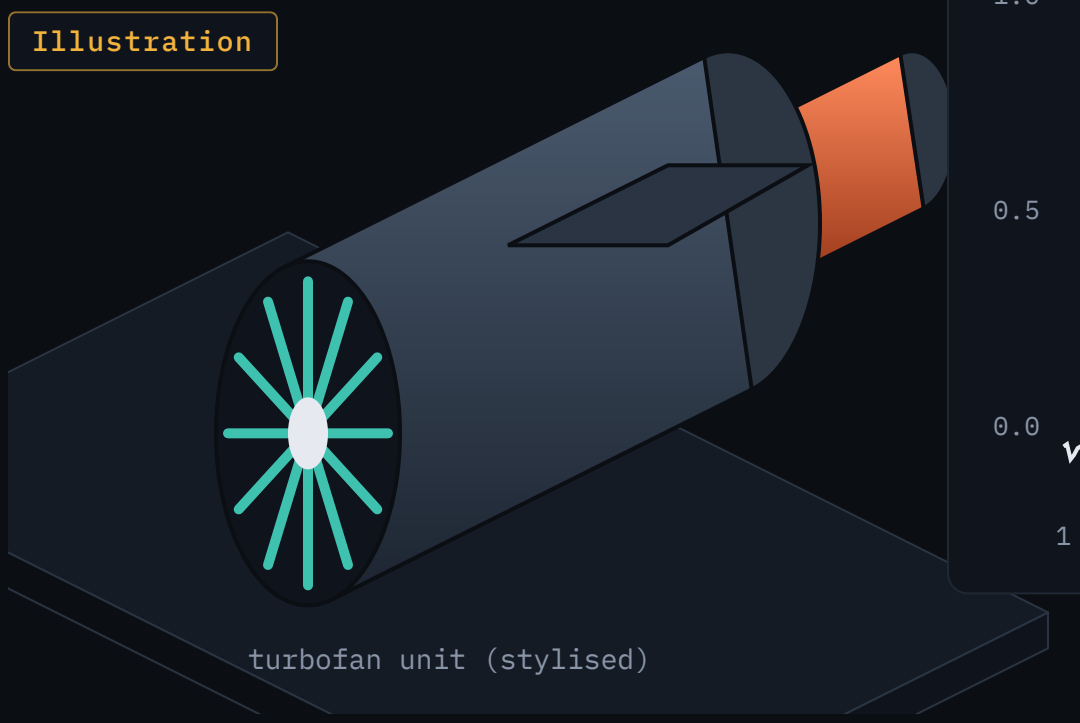
Jonathan Rincon, industrial engineer

audited portfolio prototype, Databricks Free Edition

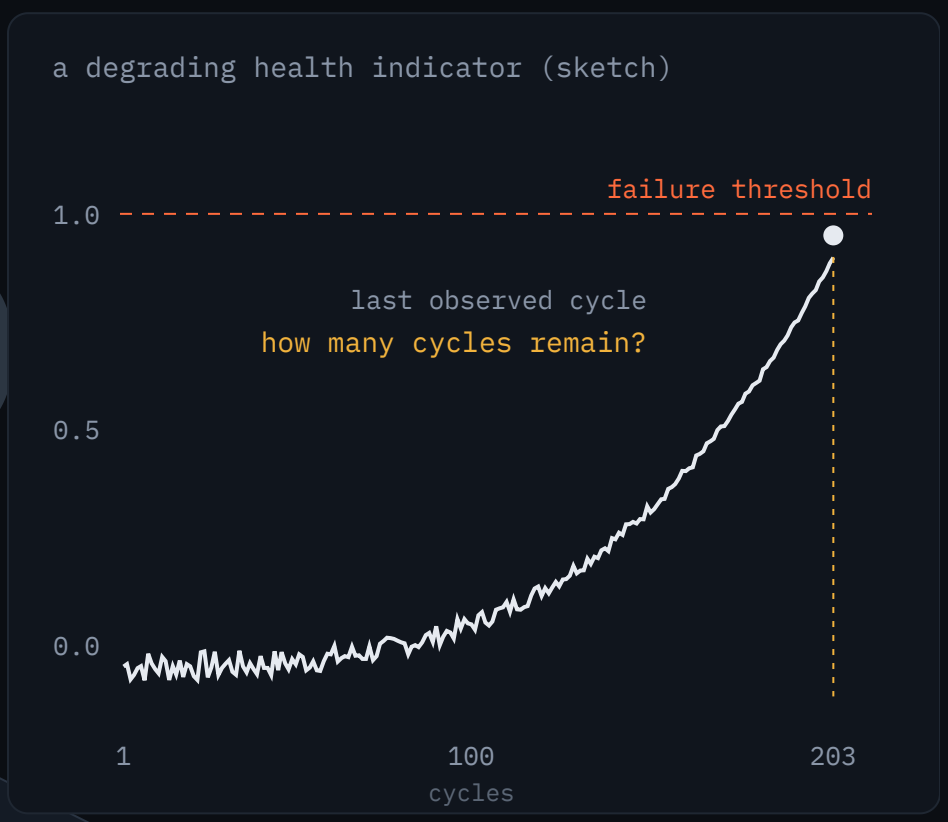
Turbofans do not fail on a schedule. They degrade.

Fixed-interval maintenance replaces healthy parts and still misses the engine that is about to fail. The question a maintainer actually asks is simpler: how many cycles are left on this unit, and how sure are we?

Illustration



turbofan unit (stylised)



THE DATA

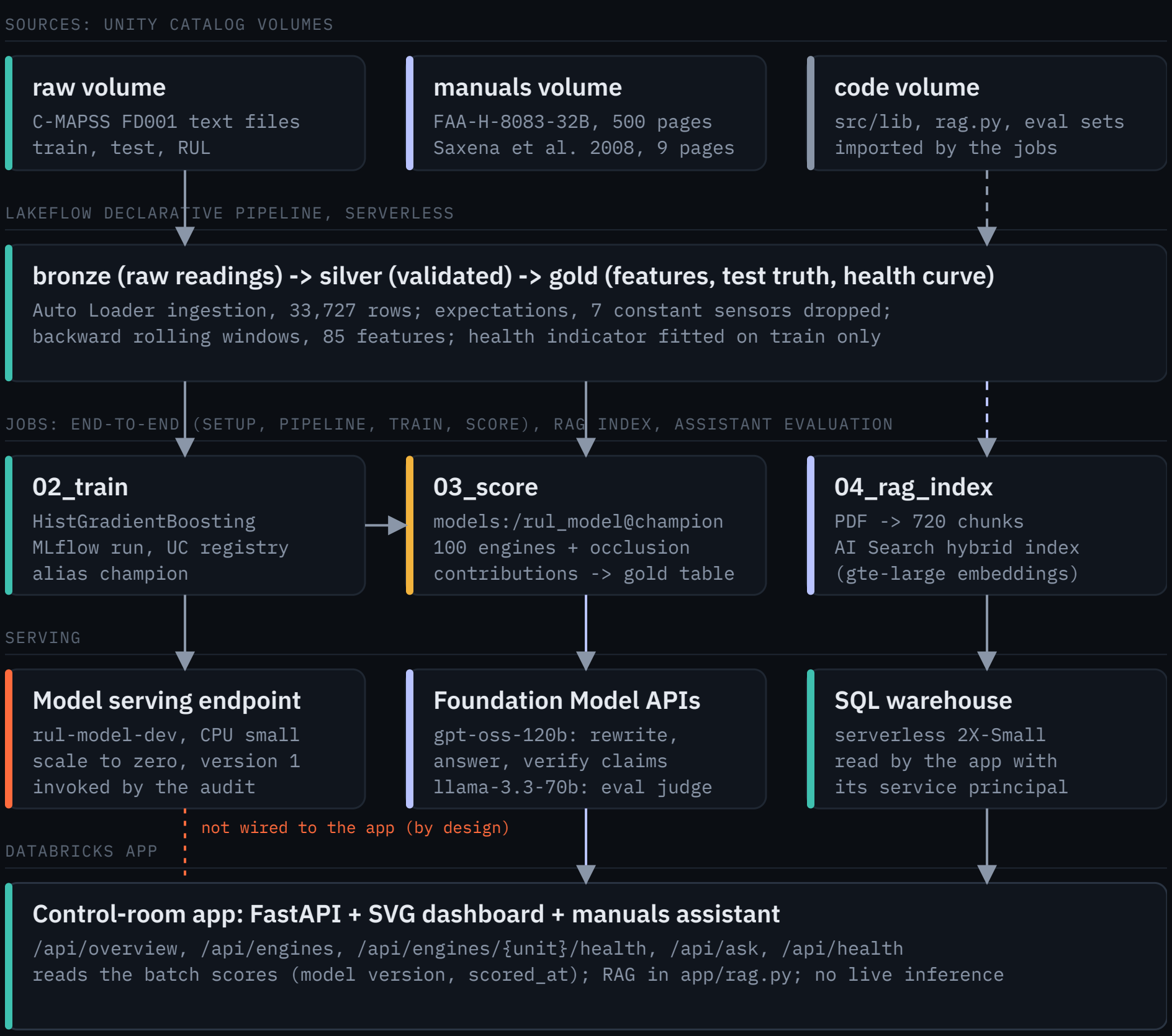
NASA C-MAPSS FD001: 100 engines run to failure for training, 100 truncated engines for testing, 21 sensors per cycle. A public simulation dataset, not fleet telemetry.

THE TARGET

Remaining useful life (RUL) in cycles at the last observed cycle of each test engine, plus a grounded answer when the maintainer asks the manual what to inspect.

One platform, five stages

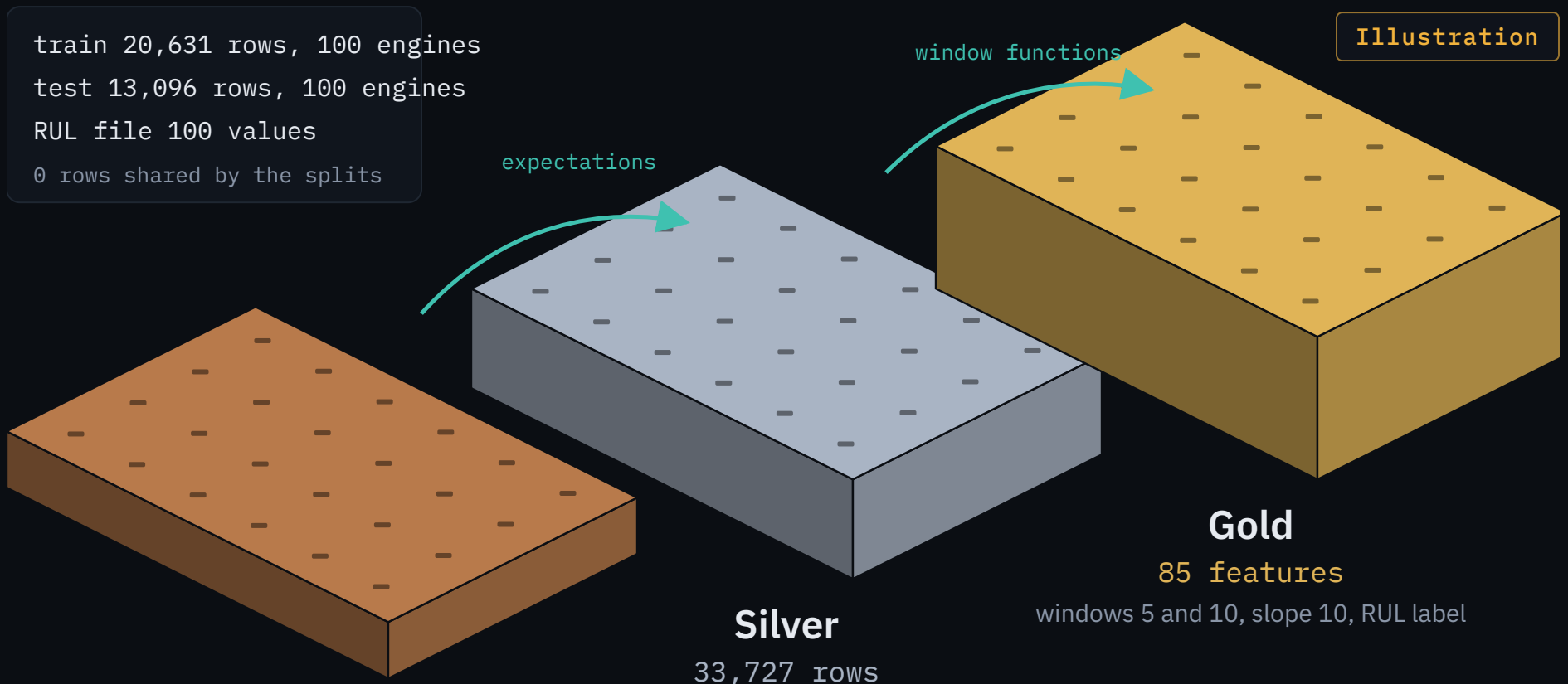
Everything runs as a Databricks Asset Bundle on serverless compute. The app reads batch scores; the serving endpoint is an independent inference interface that the audit invoked directly.



Databricks Asset Bundle, target dev, Free Edition (serverless only). GitHub Actions validates and deploys with an OAuth service principal.

Bronze, Silver, Gold: 33,727 readings become 85 features

train 20,631 rows, 100 engines
test 13,096 rows, 100 engines
RUL file 100 values
0 rows shared by the splits



WHAT THE AUDIT CHECKED

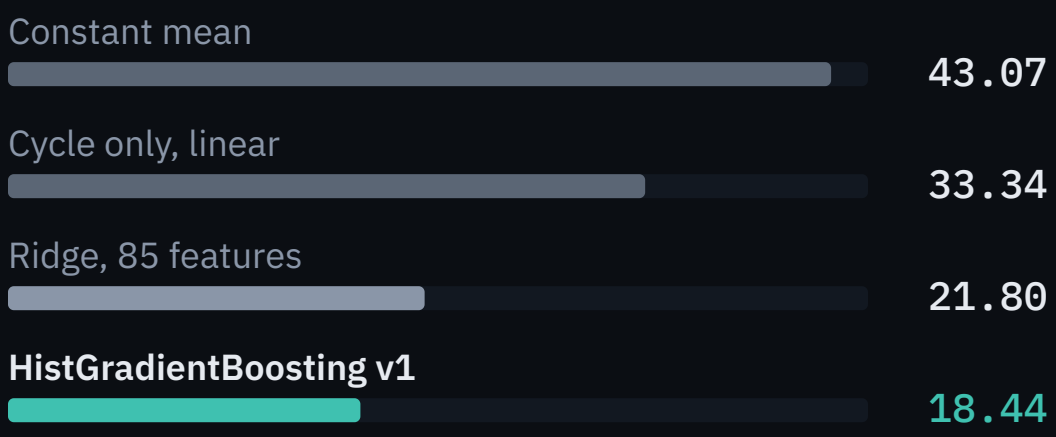
- 20,631 train and 13,096 test rows match the official FD001 files; 0 nulls, contiguous cycles.
- RUL label = last cycle minus cycle, clipped at 125: 0 discrepancies across 20,631 rows.
- Rolling windows look backward only: 40 truncation tests, 0 leaks.

PANDAS VS SPARK

- 7 constant sensors dropped, 14 kept; value, mean and std over 5 and 10 cycles, slope over 10, plus the cycle.
- 170 column sums compared between the Spark pipeline and the pandas reference: max relative difference 2.4e-9.
- Last-cycle rows of all 100 test engines: max absolute difference 3.7e-10.

RUL model: boosting beats the baselines by a measured margin

RMSE IN CYCLES, FD001 TEST, LAST CYCLE OF 100 ENGINES



ERROR BY ACTUAL RUL BAND (BIAS = PREDICTED MINUS ACTUAL)

actual RUL	engines	RMSE	bias
0 to 30	25	14.3	+5.5
31 to 60	14	8.5	+3.0
61 to 90	15	19	+5.7
91 to 125	35	17.8	-1.5
126 or more	11	32.2	-28.6

CHAMPION v1

- **HistGradientBoosting**
  scikit-learn 1.7.2, serverless job
- **85 features**
  cycle + 14 sensors x 6 statistics
- **RUL clipped at 125**
  standard practice for C-MAPSS
- **RMSE 18.44, MAE 13.48, R2 0.80**
  PHM08 655 (asymmetric late penalty)
- **Reproduced bit for bit**
  same library version, same features

HONEST READING

The test set also informed the champion rule, so 18.44 describes a frozen model on a seen set, not an independent error estimate. A by-engine protocol on train only gives 18.0 with a 1.2 spread over 200 random cuts, and the new promotion gate uses that validation (HGB 19.2 vs Ridge 22.1), never the test set.

NEAR FAILURE

With an actual RUL of 30 or less (25 engines) the model overestimates the remaining life by 5.5 cycles on average, and 6 of 26 low-RUL engines are shown as healthy. That is the unsafe direction, and the reason the dashboard labels every number a demo estimate that requires engineering review.

audit/02_MODEL_VALIDATION.md, B2

MLflow registry and serving, traced end to end

Unity Catalog model, alias champion, served on a scale-to-zero CPU endpoint. The same input row was followed through every link.

Rendered from API output — model versions

```
model      workspace.maintenance.rul_model
version    1   status READY
alias      champion -> v1
params     n_features 85
           n_train_rows 20631
           rul_clip 125
metrics    rmse_test 18.4418
           mae_test 13.4780
           phm08_test 655.02
           rmse_best_baseline 21.7963
signature  85 double inputs -> float64 (-1)
env        python 3.12.3, sklearn 1.7.2
           mlflow 3.12.0
```

Rendered from API output — endpoints get

```
endpoint dev_<user>_rul-model-dev
entity   rul_model v1
compute  CPU Small, scale to zero
state    READY

batch check, 100 engines
round(endpoint, 2) == table
  max |diff|      0.00
  mismatches      0 / 100
  RMSE vs actual  18.4418 (= MLflow)

negative inputs
  missing column  rejected
  string in s7    rejected
  nulls           accepted (D-02)
```

ONE ROW, EVERY LINK: ENGINE 34 (ACTUAL RUL 7, TEST GROUND TRUTH)

gold features	MLflow model	endpoint	scoring table	/api/engines	dashboard
engine 34, cycle 203	rul_model v1	REST invocation	batch score	app session	engine card
85 inputs	@champion	3.118922691130997	3.12	3.12	3 cycles

Same exercise for all 100 engines in one call: 0 mismatches between round(endpoint, 2) and the table.

AVAILABILITY, FREE EDITION

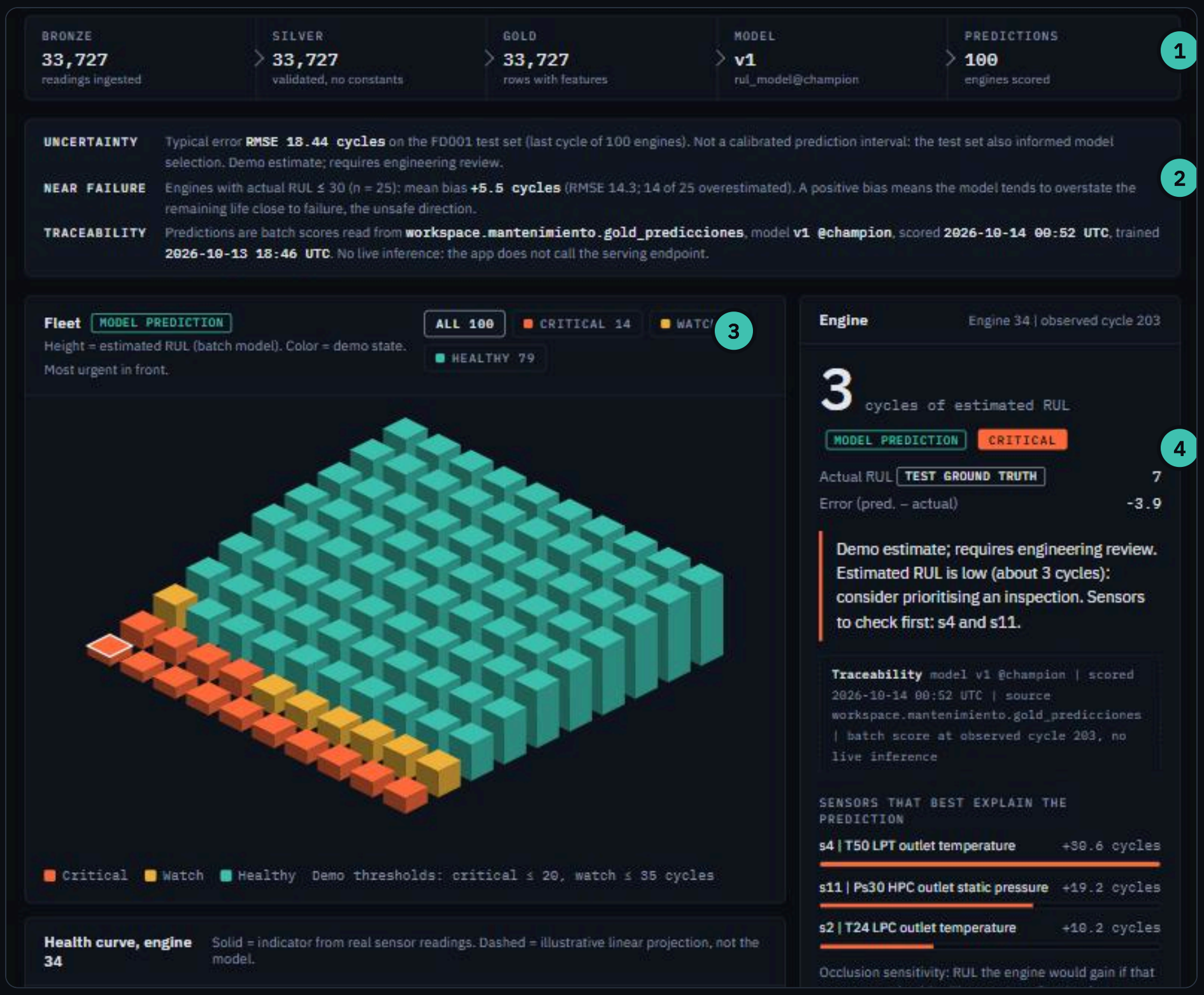
The endpoint vanished from the workspace between the two audit days and was recreated with one `bundle deploy`; the app auto-stops when idle and restarts on demand. Reproducible, not continuously available.

WHAT THE APP READS

The dashboard never calls the endpoint: it reads the batch table written by `03_score` with the same model and shows model version and scoring time. The endpoint is the independent path the audit used to cross-check every value.

A control room for 100 engines

Height is the estimated remaining life, colour is the demo state, the most urgent units sit in front. Every value is a batch score read from the gold table, with its model version and scoring time.



`Real screenshot` dev deployment, 2026-10-14, 1057 px viewport, cropped only (same UI re-verified 2026-10-15)

1 Lineage strip: bronze, silver and gold row counts, model version and engines scored.

2 Uncertainty strip: RMSE as typical error, near-failure bias, model version and scoring time.

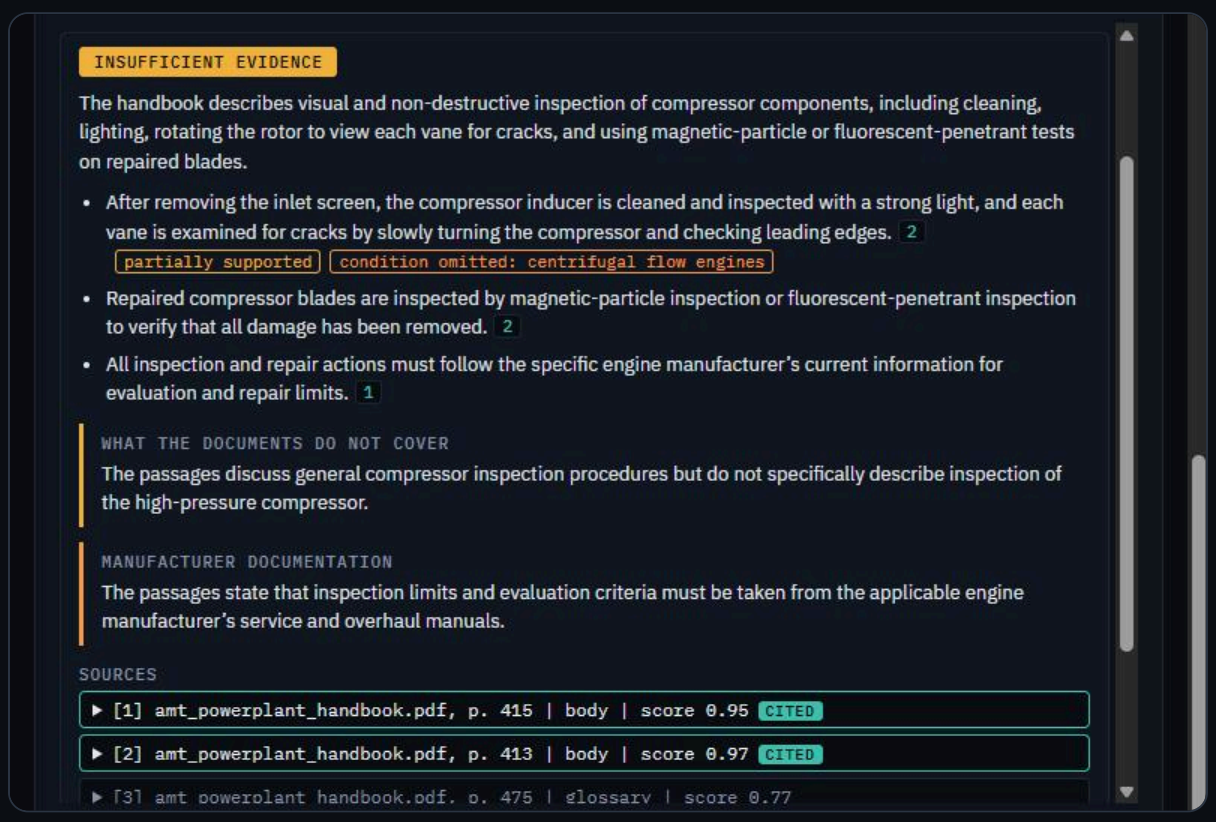
3 Filter the fleet by criticality; the isometric layout stays stable.

4 Engine 34: predicted 3 cycles, actual 7, with the sensors that drive the estimate.

A manuals assistant that shows its evidence

question any language ▸ **rewrite** EN + 3 queries ▸ **hybrid search** AI Search, RRF ▸ **structured answer** status + claims ▸ **verify each claim** judge + rules ▸ **answer** cited, with gaps

Passages and questions are treated as data: instruction-like text is stripped, numeric manufacturer values are withheld, dropped conditions are flagged.



Real screenshot dev deployment, 2026-10-14, crop of the assistant panel

Asked how the high-pressure compressor is inspected, the assistant answers `insufficient evidence`: the handbook only describes general compressor inspection. Each statement carries its page, one is flagged because the source limits it to centrifugal-flow engines, and manufacturer limits are explicitly deferred.

HELD-OUT SET: 85 NEW QUESTIONS, 3 LANGUAGES

97.6%	correct status (v1 baseline 78.8%)
0	unsupported shown claims, two judges (v1: 62.6%)
98.9%	citation precision
0.84	MRR; hit@6 0.97; page recall 0.89
100%	appropriate abstention
14.6 s	mean latency, 5.5 LLM calls per question

MANDATORY TEST: HIGH-PRESSURE COMPRESSOR INSPECTION

The handbook has no HPC-specific procedure, only generic compressor-section inspection (pp. 413 to 415). In 9 of 9 reviewed answers the assistant did not turn that generic text into an HPC procedure; it abstained or declared the gap. Re-asked on the live app on 2026-10-15: `not_found` (Spanish and English wording) and `insufficient_evidence` with the omitted condition flagged (UI preset). Answers vary between runs: a known, documented limitation.

What is verified, and what is not

VERIFIED

- Data counts, labels and windows match FD001; Spark and pandas features agree to 1e-9.
- Champion v1 reproduced bit for bit (RMSE 18.441849882005343) with the workspace library versions.
- Endpoint invoked for all 100 engines: identical to the dashboard table, 0 mismatches.
- No silent fallback: SQL failure is an explicit 503, a null prediction stays null.
- RAG: 85-question held-out set, deterministic checks plus an independent judge model.
- 142 automated tests, pip-audit and bandit clean, no secrets in tree or history.

LIMITATIONS

- The FD001 test set is a seen set: it informed model selection, so no independent error estimate exists yet.
- Uncertainty is not calibrated: RMSE is a typical error, not a prediction interval.
- Near failure the model overestimates life by 5.5 cycles on average; 6 of 26 low-RUL engines show as healthy.
- Predictions are batch scores at the last observed cycle, not live inference on telemetry.
- The serving endpoint accepts null features without error (open finding, medium).
- Assistant answers take 10 to 30 s and vary between runs at temperature 0.1.

18.44
RMSE in cycles on the seen FD001 test set

0 / 100
endpoint vs dashboard mismatches

97.6%
correct assistant status, 85 held-out questions

142
automated tests passing, security scans clean

A simulation dataset and a demo model. This is a portfolio-grade, audited prototype, not a certified aeronautical maintenance system.

Built in the open, with the receipts

Jonathan Rincon (industrial engineer) defined the architecture, the technical parameters and the implementation sequence, and reviewed every phase of the work.

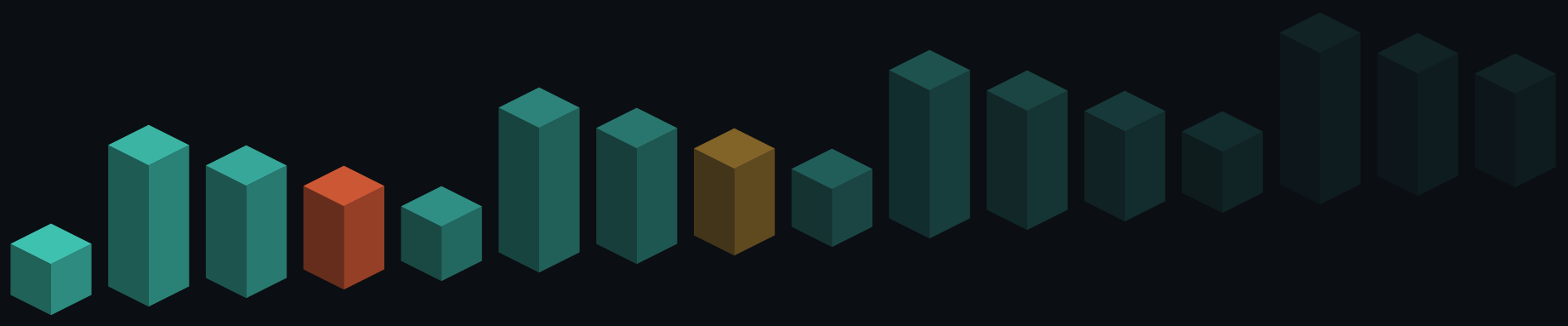
STACK

- Databricks Asset Bundles
- Lakeflow Declarative Pipelines
- Unity Catalog
- Delta Lake
- Auto Loader
- Spark SQL
- pandas
- scikit-learn
- MLflow
- Model Serving
- AI Search (vector)
- Foundation Model APIs
- Databricks Apps
- FastAPI
- SVG, no framework
- pytest (142)
- GitHub Actions
- Playwright

REPOSITORY

github.com/jonathanjrm/databricks-mantenimiento-predictivo

Audit reports, evidence files, evaluation sets, tests and this carousel's source are part of the project tree and are published with the reviewed branch.



Illustration

NASA C-MAPSS data, public domain